共用环境部署与操作手册

本手册汇总了项目三个阶段所涉及的工程目录结构、软件工具链配置、脚本执行入口及结果验收准则。除另有说明外，所有脚本与测试命令均以相关证明材料作为工作根目录运行；关于指令架构定义、微架构实现及各项实测数据分析，请分别查阅对应的专项技术报告。在实际操作中，建议先执行第 3 章的本地自动化核验，确认环境与归档数据无误后，再参照第 6 章开展完整处理器的编译构建与协同仿真。

1 目录与使用方式

操作时请保持证明材料内部子目录层级完整，各核心子目录的工程定位如下表所示。

位置	内容与用途
stage1	阶段一环境信息输出程序
sources 与 source_additions	固定版本香山、NEMU 及所需子工程源码
rtl 与 nemu	点积硬件电路、香山后端接入补丁及 NEMU 扩展实现
tests、programs 与 examples	939 组测试向量、32 个处理器测试程序及最小调用示例
recorded 与 results/cluster/20261019	完整处理器验证记录与周期延迟实测数据集
waveforms/vadd.fst	原始完整处理器向量加法波形
results/vadd_wave_stage1	波形导出表与指令执行分析
scripts、tools 与 cluster	分项检查、构建、执行和分析程序

本手册给出的所有命令均默认在相关证明材料根目录下执行。在 Windows 环境下，可在资源管理器打开该文件夹后直接在地址栏输入 powershell 并回车启动终端；在 Linux 环境下，可通过终端切换至该目录。除第 2 章指定的工具安装路径外，其余运行分析与重算产生的输出文件均须指定至材料目录外部的新建目录或空目录，以免污染原始交付材料。

2 软件与环境准备

2.1 本地检查所需软件

软件	要求	用途
Python	3.10 或更新版本	执行交付材料完整性核验、归档结果离线复算与统计分析
Python 依赖	requirements-local-check.txt	支撑波形文件解析与环境自检，版本约束需与 requirements.txt 保持一致
Icarus Verilog	终端可直接调用 iverilog 与 vvp	运行 Chisel 与 Verilog 两套独立点积算术电路的本地功能仿真
RISC-V GNU 工具	as、ld、objcopy、objdump、nm	对汇编示例程序与分类测试基准程序执行本地重新汇编与链接
FST 读取库	Windows 附 tools/fsttool/fstapi.dll	直接读取原始波形；Linux 需编译对应共享库

```
python -m pip install -r requirements-local-check.txt
```

自动化总检脚本会自动在系统路径中检索 RISC-V 交叉编译工具链。若本机工具链安装在非标准自定义路径下，需在执行命令末尾追加 --prefix 参数并传入完整前缀路径。执行汇编重编译检查时，需确保 as、ld、objcopy、objdump 和 nm 五个工具组件均正常可用。

2.2 本地 Linux 工具与超算使用约定

本项目的香山完整处理器构建与全套协同仿真均在超算 Linux 计算节点上完成；本地 Windows 及 WSL 环境主要用于前期工具准备、测试程序交叉编译及离线数据复核。以下安装流程针对本地 Ubuntu 环境编写；在超算集群上运行时，应直接调用用户目录内预置的同版本工具链或集群公共模块，切勿在超算登录节点直接运行需要 root 权限的系统级安装指令。尚未配置 WSL2 的 Windows 主机，可先以管理员身份打开 PowerShell 执行以下命令，并按终端提示重启系统及初始化 Ubuntu 账户：

```
wsl --install -d Ubuntu-24.04
```

进入 Ubuntu 终端后，先切换至解压后的相关证明材料目录，再执行如下命令将整套材料完整拷贝至 Linux 用户工作区：

```
mkdir -p "$HOME/vdot-work"
test ! -e "$HOME/vdot-work/materials" || exit 1
cp -a "$PWD" "$HOME/vdot-work/materials"
cd "$HOME/vdot-work/materials"
```

配套的工具链安装脚本会调用系统包管理器安装基础编译组件、JDK 17 与 Icarus Verilog，并自动拉取固定版本的 Mill 0.12.3、firtool 1.62.1、Verilator 5.052 以及 xPack RISC-V GCC 14.2.0-3。该过程依赖外网连接与 sudo 权限，且源码下载及 Verilator 本地编译耗时较长：

```
export VDOT_TOOLS="$HOME/vdot-work/tools"
export DOWNLOADS="$HOME/vdot-work/downloads"
bash scripts/install_tools.sh
source scripts/env.sh
python3 -m venv "$HOME/vdot-work/python-env"
source "$HOME/vdot-work/python-env/bin/activate"
python3 -m pip install -r requirements-local-check.txt
python3 -B tools/fsttool/build_reader.py --out tools/fsttool/libfstapi.so
```

当终端最终打印 TOOLS_INSTALL_PASS 时，表明工具链部署成功。在首次安装完毕后，以及后续每次新开 Linux 终端窗口时，均需按第 2.3 节流程加载工具环境变量并激活 Python 虚拟环境。

FST 波形读取动态库默认编译输出至 tools/fsttool/libfstapi.so，单次编译后即可长期复用；若该文件已存在，后续可跳过编译步骤。若需在同一环境内并存多个版本的动态库，可在编译时通过 --out 指定新的输出文件名，并在调用 reproduce_vadd.py 时通过 --library 参数显式指定目标库路径。

第 6 章的完整处理器构建默认采用 Java 最大堆内存 64G、2 路并行编译任务的参数配置。在超算平台提交作业时，可参考 cluster/build.sbatch 脚本中 16 核 CPU、90000 MB 内

存的资源配额进行申请。考虑到宿主机还需承载操作系统开销及 Verilator 的 C++ 多线程编译峰值，节点实际可用物理内存须高于所设定的 Java 最大堆上限。

工程构建所使用的 RTL 与模拟器源码均以 sources 和 source_additions 目录内随附的固化版本为准，各组件对应的精确 Commit 版本号记录于 source_revisions.csv。请勿直接拉取开源社区主分支的最新代码替换随附源码包。相关上游开源仓库地址仅供对照参考：github.com/OpenXiangShan/XiangShan；

github.com/OpenXiangShan/NEMU；

github.com/OpenXiangShan/XiangShanLab

2.3 每次打开 Linux 终端后的环境设置

完成第 2.2 节的初始化部署后，每次新开启 Linux 终端会话时，请先依次执行以下命令加载环境并核验工具链版本。正常情况下，终端应分别回显 Java 17、Mill 0.12.3、firtool 1.62.1、Verilator 5.052 以及 RISC-V GCC 14.2.0：

```
cd "$HOME/vdot-work/materials"
export VDOT_TOOLS="$HOME/vdot-work/tools"
source scripts/env.sh
source "$HOME/vdot-work/python-env/bin/activate"
python3 --version
java -version
mill --version
"$CHISEL_FIRTOOL_PATH/firtool" --version
verilator --version
riscv64-unknown-elf-gcc --version
```

3 本地配套程序检查

若需对交付包内的配套脚本与离线数据进行全量一键自检，可执行以下命令。脚本默认将检查报告写入系统临时目录并在终端打印具体输出路径；若需自定义保存位置，可通过 --out 指定材料目录以外的新目录或空目录：

```
python -B check_programs.py
```

总检程序检查运行环境、939 组输入、随附处理器结果、数学参考模型、两种硬件电路、数据处理、向量加法波形、输入处理和示例重编译，共 12 项。全部通过时，终端输出 COMPLETE passed=12 failed=0 unavailable=0。当前 252 次性能结果按第 7 章单独复算。配套检查输出明细保存在终端显示的结果目录内。

本地检查读取随附结果并执行独立电路与编译检查。重新启动完整处理器使用第 6 章，当前性能程序使用第 9 章。基础输入和处理器结果可分别运行：

```
python -B scripts/check_materials.py
python -B scripts/check_recorded_results.py
```

两项检查通过后，终端将分别打印 MATERIALS_PASS cases=939 groups=4 与

RECORDED_RESULTS_PASS。其中，第二个检查入口会严格比对 NEMU 的 128 位整寄存器参考结果、96 项封装层配置检查记录、32 个处理器测试程序记录、原始测试激励以及 19 份核心源码文件的一致性。

4 阶段一输出与向量加法

4.1 输出个人标识和当前 IP

阶段一示例程序通过读取环境变量 TEAM_ID 获取团队标识。在 Windows PowerShell 终端下，请依次执行：

```
$env:TEAM_ID="GEMwhu"
python -B stage1/hello_xiangshan_anyos.py
.\stage1\hello_xiangshan.exe
```

在 Linux 终端下，可先运行 Python 脚本，再调用本机 GCC 编译 C 语言版本并执行：

```
TEAM_ID=GEMwhu python3 -B stage1/hello_xiangshan_anyos.py
gcc -O2 stage1/hello_xiangshan.c -o "$HOME/vdot-work/hello_xiangshan"
TEAM_ID=GEMwhu "$HOME/vdot-work/hello_xiangshan"
```

程序正常运行时，首行输出格式应为 hello xiangshan, I am GEMwhu, IP address，其后紧跟当前运行主机的实际网卡 IP 地址。关于香山处理器内部执行向量指令的波形分析与协同验证结果，请分别参阅第 4.2 节与第 6 章。

4.2 复查原始向量加法波形

```
python -B reproduce_vadd.py --mode fst
```

执行上述命令将直接解析 waveforms/vadd.fst 原始波形文件，在终端打印向量加法指令的流水线执行轨迹、寄存器写回数据及自动比对结论。随附波形文件完整记录了 11 次向量加法运算及对应的 176 字节输出数据，详细的逐拍流水分析参见《1.3_向量加法指令执行分析报告》。若需基于随附的 CSV 导出表进行分析，可将参数切换为 --mode csv；若需同步生成可视化图表，可追加--plot 参数。

若需对原始向量加法汇编测试程序进行重新交叉编译，可调用 build_vadd.py 入口，并显式传入 RISC-V 工具链前缀及位于材料目录外部的新输出路径：

```
python -B build_vadd.py --prefix riscv64-unknown-elf- --out ../vadd-build-new
```

若需验证波形分析脚本对损坏或非法输入的容错拦截能力，可执行下述独立自检命令；若需在完整处理器上重新运行并录制新的向量加法波形，请按第 6.2 节流程操作。

```
python -B reproduce_vadd.py --mode invalid
```

5 基础功能与测试程序

5.1 本地电路检查

```
python -B scripts/check_basic.py
```

该脚本会将 46 组正数用例、48 组负数用例、45 组边界值用例以及 800 组随机用例分别灌入独立 SystemVerilog 点积模块与 Chisel 生成的 Verilog 点积电路中开展仿真，并对每

组输出的完整 128 位结果进行逐位比对。当两套电路全部通过 939 例验证时，终端输出 BASIC_TEST_PASS cases=939 implementations=2。关于这批用例在香山完整处理器上的运行方式，详见第 5.2 节与第 6 章；正数、负数与边界值三类专项用例的构造说明与预期输出详见对应的三份测试文档（2.3.1 至 2.3.3）。

5.2 在同一个完整处理器构建中执行分类测试

开展处理器级分类回归前，需先按第 6.1 节完成香山完整处理器构建，并在当前终端会话中保持 RUN_ROOT 变量有效。随后执行以下两条命令，自动生成正数、负数、边界值三个分类测试汇编程序及索引表 programs.csv，并调用工作区内的香山仿真器 emu 与 NEMU 动态参考库完成这三组程序的协同仿真：

```
python3 -B scripts/prepare_tests.py --out "$RUN_ROOT/programs"
python3 -B scripts/run_tests.py --run-root "$RUN_ROOT" --group all
```

三组分类测试通过后分别输出 BASIC_TEST_PASS。逐程序输出位于 RUN_ROOT/basic_results，汇总表为 latest_results.csv。单类参数可用--group pos、--group neg、--group bound；综合程序使用--group corpus。

5.3 单个程序与新增输入

若需在已构建的处理器工作区内单独调试或运行某个特定测试程序，可执行以下命令：

```
python3 -B scripts/run_one.py --run-root "$RUN_ROOT" --program minimal_vdot
```

运行通过后，终端将打印 SINGLE_PROGRAM_PASS 及预设的成功退出 PC 地址，完整仿真记录保存于 RUN_ROOT/single_program_results/minimal_vdot_emu.txt。如需运行其他专项功能测试程序，可从 programs/manifest.json 清单中查询对应程序名并通过 --program 参数传入；该入口同样复用同一套香山仿真器与 NEMU 参考库。

若需扩展自定义测试向量，建议在独立拷贝的开发工作区中修改 tests 目录下的激励文件，并同步更新测试程序生成逻辑、用例计数及期望真值表，随后重新执行汇编与处理器仿真。鉴于随包附带的自检脚本默认按固化的 939 例规模做完整性校验，新增扩展用例产生的测试结果请单独归档保存。

6 完整处理器构建与协同验证

6.1 一次构建供所有阶段使用

本节构建命令需在已分配充足计算资源的超算 Linux 节点上运行。操作前请先按第 2.3 节加载工具链环境并进入相关证明材料根目录，随后指定一个尚未创建的绝对路径作为 RUN_ROOT。构建脚本将自动完成固定版本源码拷贝、vdot.vv 硬件与模拟器补丁合入、NEMU 动态参考库编译、香山处理器 Verilator 仿真器 emu 编译以及全部 32 个裸机测试程序的交叉编译：

```
source scripts/env.sh
export RUN_ROOT="$HOME/vdot-work/full-new"
export BUILD_JOBS=2
export EMU_COMPILE_JOBS=2
python3 -B run_latency.py check
python3 -B run_latency.py build --work "$RUN_ROOT" --jobs 2 --java-heap 64G
python3 -B cluster/build_manifest.py
```

当整套编译流程顺利结束时，终端将输出 BUILD COMPLETE。核心构建产物包括香山仿真器可执行文件 RUN_ROOT/XiangShan/build/emu、NEMU 参考动态库 RUN_ROOT/NEMU/build/riscv64-nemu-interpreter-so 以及测试程序镜像目录 RUN_ROOT/programs。末尾调用的 cluster/build_manifest.py 脚本负责读取 SUBMISSION 与 RUN_ROOT 信息并生成构建清单，供第 6.2 节协同验证调用；整个构建过程配置 Java 最大堆内存为 64G。

6.2 完整处理器与 NEMU 同步执行

```
python3 -B scripts/run_full.py --run-root "$RUN_ROOT" --workers 4 --repeat 3
```

该入口启动香山处理器与 NEMU 同步执行随附基础程序集，并录制向量加法 FST 波形。结果位于 RUN_ROOT/validation，成功提示为 FULL_SOC_PASS。当前性能程序和数据分析统一使用第 7、9 章入口。

在协同验证结果判定上，脚本不仅要求仿真进程返回码为 0 且 DiffTest 全程无状态失配，还实施了严格的指令流终点校验：所有功能验证程序必须准确停机在预设的 OK 成功地址，所有性能测速程序必须停机在合法的计时采样地址区间内；凡出现未预期异常、主从处理器状态失配、运行超时或跳入失败分支等任一情形，均严格判定为测试失败。关于 128 位整寄存器向量状态的协同对比机制，详见阶段三报告《3.1_NEMU 指令扩展与协同仿真验证报告》。

分类功能检查、完整处理器执行和当前性能程序均复用 RUN_ROOT 中的 emu 与 NEMU 参考库。每次执行使用独立结果目录，便于查阅本次输出。

7 处理器计时结果复算

当前性能结果为 252 次计时：通用矩阵接口 12 个程序各三次，共 36 次；独立点积 72 个程序各三次，共 216 次。下列命令读取随附数据并重新计算统计结果，使用 Python 标准库，不启动处理器：

```
python -B scripts/check_measurements.py --out ../measured-check-new
```

成功提示为 MEASURED_RESULTS_PASS records=252 groups=84 expected_repeats=3。输出目录包含矩阵和独立点积两项检查的明细。程序同时核对三次重复、结束地址、8 周期区间、输入来源及分析表数值。

16 个独立点积的含循环周期为标准 RVV 5.022429、自定义指令 1.064012 周期/点积，周期比 4.720。32 方阵含 B 整理的直接取回与四列方法分别为 31984、23032 周期，四列减少 27.989%。完整定义、各尺寸和曲线见 3.3 报告。

8 结果的保存位置

内容	保存位置
本地配套检查	--out 指定目录，或终端显示的临时目录
基础处理器结果	results/cluster/20261019；第 3 章脚本读取核对
向量加法波形	waveforms/vadd.fst；导出表 results/vadd_wave_stage1
当前矩阵计时	applications/gemm/measured_20261026
当前独立点积计时	experiments/throughput/measured_20261026
任务与资源	measurements/20261026

9 矩阵应用、独立点积与补充检查

以下命令从相关证明材料根目录执行，结果使用工程以外的新目录。Linux 命令中的 RUN_ROOT 沿用第 6 章构建目录。

9.1 本地功能检查

```
python -B scripts/check_improvements.py --out ../improvements-check-new
```

该入口执行矩阵接口、独立点积程序和补充电路检查。各项成功提示及所需软件见对应 README。已有处理器周期直接使用第 7 章命令复算。

9.2 通用矩阵接口

```
python -B applications/gemm/check.py --out ../gemm-check-new
python -B applications/gemm/check_measured.py --out ../api-measured-check-new
```

接口定义位于 applications/gemm/include/vdot_gemm.h，返回 C_REFERENCE 或 CUSTOM_VDOT。直接取回和四列方法均支持 K 尾部补零、N 尾列及 K=0。已测程序覆盖三种尺寸、两种方法、两种计时范围，每组三次。

按当前接口生成并交叉编译 12 个程序，随后使用第 6 章的处理器执行 36 次计时：

```
export API_PROGRAMS="$HOME/vdot-work/api-programs-new"
export API_RESULTS="$HOME/vdot-work/api-results-new"
python3 -B applications/gemm/api_benchmark.py generate --out "$API_PROGRAMS"
python3 -B applications/gemm/api_benchmark.py execute \
  --programs "$API_PROGRAMS" --emu "$RUN_ROOT/XiangShan/build/emu" \
  --library "$RUN_ROOT/NEMU/build/riscv64-nemu-interpreter-so" \
  --out "$API_RESULTS" --repeat 3 --workers 16 --seconds-limit 7200
python3 -B applications/gemm/api_benchmark.py analyze \
  --results "$API_RESULTS" --out "$API_RESULTS/recomputed" --repeat 3
```

generate 同时生成和编译程序；execute 完成处理器执行并自动分析，程序周期上限读取 programs.csv；analyze 可独立重算。执行所用并发数应与实际分配资源一致，示例使用 16 核。

9.3 独立点积数量比较

```
python -B experiments/throughput/check.py --out ../throughput-check-new
```

```
python -B experiments/throughput/check_measured.py --out ../throughput-measured-check-new
```

study.py 生成 1、2、4、8、16 个独立点积及原址依赖链，包含标准 RVV、自定义指令与空循环。N 为 0、64、256、1024，共 72 个程序。重新执行命令如下：

```
export DOT_PROGRAMS="$HOME/vdot-work/dot-programs-new"
export DOT_RESULTS="$HOME/vdot-work/dot-results-new"
python3 -B experiments/throughput/study.py generate --out "$DOT_PROGRAMS"
python3 -B experiments/throughput/study.py estimate \
  --programs "$DOT_PROGRAMS" --repeat 3 --workers 16 \
  --cycle-limit 250000 --seconds-limit 7200
python3 -B experiments/throughput/study.py execute \
  --programs "$DOT_PROGRAMS" --emu "$RUN_ROOT/XiangShan/build/emu" \
  --library "$RUN_ROOT/NEMU/build/riscv64-nemu-interpreter-so" \
  --out "$DOT_RESULTS" --repeat 3 --workers 16 \
  --cycle-limit 250000 --seconds-limit 7200
python3 -B experiments/throughput/study.py analyze \
  --results "$DOT_RESULTS" --out "$DOT_RESULTS/recomputed"
```

本次任务以 16 核完成上述 84 个程序、252 次计时，总用时 9 小时 51 分 48 秒。已完成结果可直接读取复算，任务资源记录位于 measurements/20261026。

9.4 补充电路与机器指令检查

```
python -B verification/check.py --out ../position-check-new
```

48 组输入检查两种电路完整 128 位输出，成功识别 41 种指定错误并确认 28 种双输入同步交换，共 3408 项比较。coverage.csv 说明各类输入与检查目标。机器指令检查读取编译产物，随附最小程序可使用：

```
python -B verification/check_instructions.py examples/minimal_vdot.elf --out ../instruction-check-new
```

需要标量取回的其他程序可追加--require-extract，参数说明见 verification/README.md。源目的重合、向量配置及非法指令异常由完整处理器专项程序检查。

10 当前性能图表生成

图表直接读取通用矩阵 36 次测量统计及独立点积 216 次测量分析表。图中使用与 3.3 报告一致的当前数据和计时范围。

```
python -m pip install -r performance/requirements.txt
python -B performance/build_figures.py
```

矩阵图读取 applications/gemm/measured_20261026/summary.csv；独立点积曲线读取 experiments/throughput/measured_20261026/analysis 下的 comparison.csv 和 speedup.csv。输出 PNG、SVG 和 PDF 位于 performance/figures。